INSTITUTO FEDERAL
RIO GRANDE DO SUL

Estruturas de Dados II

Alocação Dinâmica de Memória

Uso de Ponteiros

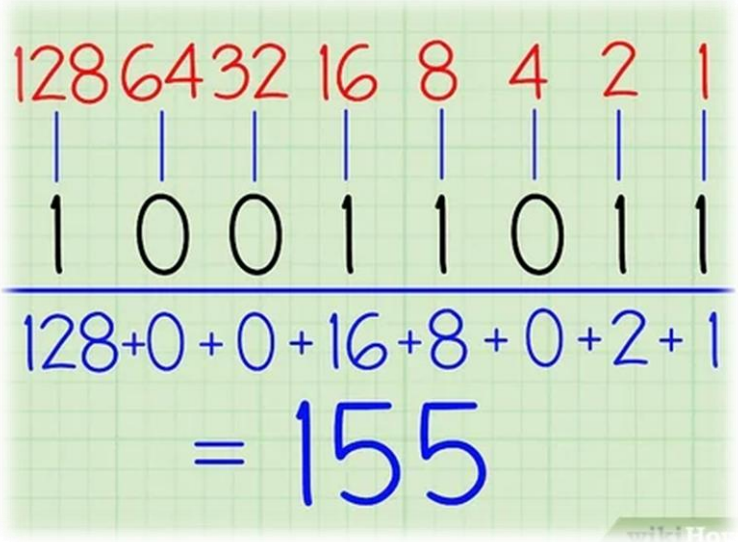
RELEMBRANDO... SESSÃO REVISÃO

SISTEMA BINÁRIO

Como o próprio nome já indica, tem base 2 e é o sistema de numeração mais utilizado em processamento de dados digital, pois utiliza apenas dois símbolos ou algarismos 0 e 1.

Também vale ressaltar, que em processamentos digitais, que o dígito 1 também é conhecido por nível lógico 1, nível lógico alto, ligado, verdadeiro e energizado.

Já o dígito 0 poder ser nível lógico 0, nível lógico baixo, desligado, falso ou sem energia.



RELEMBRANDO... SESSÃO REVISÃO

Por que os caracteres consomem 1 byte?

Todos os caracteres representáveis na memória foram codificados em uma tabela denominada Tabela **ASCII (American Standard Code InterchangeInformation).**

Originalmente a Tabela ASCII possuía apenas 128 códigos que representavam o alfabeto, os caracteres de pontuação e alguns caracteres de controle do idioma inglês. Atualmente, além dos 128 códigos originais, possui mais 128 códigos adicionais para um conjunto de caracteres denominado caracteres estendidos.

Assim, cada tecla do teclado tem um código ASCII associado a ela, de modo que, quando uma tecla é digitada, uma posição de memória ou 1 B da memória será preenchido com o seu código correspondente.

RELEMBRANDO... SESSÃO REVISÃO

Por que um inteiro ocupa 4 bytes?

Da mesma forma, o primeiro bit é usado para o sinal, e os 31 bits restantes para armazenar o valor do número na base dois.

[s][b][b][b][b][b][b][b] [b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b][b]

Limitação: Neste caso, o valor do número inteiro será armazenado usando 31 bits. Portanto:

o maior inteiro longo será 2 a 31 -1 = + 2.147.483.647
o menor inteiro longo será -2 a 31 = - 2.147.483.648

RELEMBRANDO... SESSÃO REVISÃO

Alocação dinâmica de memória - Visão mais detalhada da memória do computador

Considere o caso **int x = 9;**

O x poderia ir parar à posição de memória 0xE2 quando o programa fosse executado. **Na realidade, as coisas não são bem assim.**

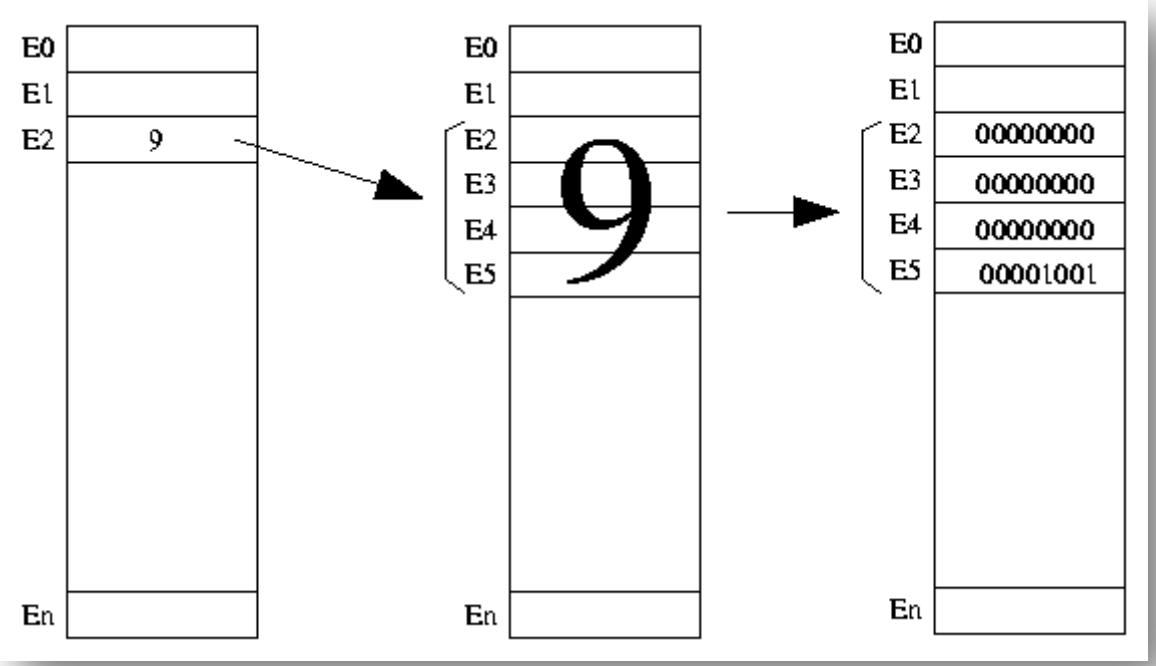
Cada posição de memória corresponde apenas a um byte e uma variável inteira ocupa geralmente 4 bytes.

Ou seja, o número 9 não vai estar representado numa única posição de memória mas sim em 4 posições de memória (ex: E2, E3, E4 e E5).

Aliás, aquilo que está nessas 4 posições de memória é a representação em binário do número 9.

RELEMBRANDO... SESSÃO REVISÃO

Apesar da variável x estar localizada nos endereços E2, E3, E4 e E5, costuma dizer-se que o endereço da variável x é E2. Isto é, o endereço de uma variável é o endereço do primeiro byte que a variável ocupa.



RELEMBRANDO... SESSÃO REVISÃO

E os arrays? Como é que são guardados?

Se declararmos um array a de 10 inteiros, o compilador vai reservar um bloco de memória consecutivo que permita guardar esses 10 inteiros.

Se um inteiro ocupar 4 bytes, o compilador terá de reservar um bloco de 40 bytes (por exemplo, do endereço E100 até ao endereço E139):

```
a[0]  vai ocupar as posições E100, E101, E102, E103
a[1]  vai ocupar as posições E104, E105, E106, E107
a[2]  vai ocupar as posições E108, E109, E110, E111
...
a[9]  vai ocupar as posições E136, E137, E138, E139
```

INSTITUTO FEDERAL
RIO GRANDE DO SUL

REFERÊNCIA

Código Fonte

```
"#include <stdio.h>

int main() {
   char letra = 'A'; //0x01A
   int num = 1500; //0x01E
   double pi = 3.1415; //0x00F

   printf (""\n%d "", sizeof(letra));
   printf (""\n%d "", sizeof(num));
   printf (""\n%d "", sizeof(pi));


   printf (""\n%d "", letra);
   printf (""\n%c "", letra);

  return 0;
}"
```

Em Geral

inteiros = 4 bytes
float = 4 bytes
double = 8 bytes
char = 1 byte

Endereço	Memória
0x00F	0000000
0x010	0110001
0x011	0100001
0x012	11001010
0x013	11000000
0x014	10000011
0x015	00010010
0x016	01101111
0x017	
0x018	
0x019	
0x01A	01000001
0x01B	
0x01C	
0x01D	
0x01E	00000000
0x01F	00000000
0x020	00000101
0x021	11011100
0x022	
0x023	
0x024	
0x025	

RELEMBRANDO... SESSÃO REVISÃO

Do mesmo modo que dizemos que o endereço da variável x é E2, podemos dizer que o endereço do array a é E100.

Isto é, o endereço do array é o endereço do primeiro byte que o array ocupa.

De fato, quando declararmos:

```
int a[10];
```

O nome "**a**" é o endereço da primeira posição do array.

Por outras palavras, "a" é sinônimo de &a[0] . É perfeitamente válido fazer coisas deste estilo,

```
int a[10];
int *p;
p = a;
*p = 10;
```

/* equivalente a dizer a[0] = 10 */

REFERÊNCIA

Referência é quando nos referimos diretamente ao identificador do endereço da memória.

A memória de um computador é na verdade uma grande tabela com células sequenciais,cada célula tem seu próprio endereço que segue um padrão contínuo.

REFERÊNCIA

Podemos considerar char a[8],

a primeira célula	0x00F,	Conteúdo [00000000]
...a segunda	0x01F,	Conteúdo [00000000]
...a terceira	0x02F,	Conteúdo [00000000]
...a quarta	0x03F,	Conteúdo [00000000]
...a quinta	0x04F,	Conteúdo [00000000]
...a sexta	0x05F,	Conteúdo [00000000]
...a sétima	0x06F,	Conteúdo [00000000]
...a oitava	0x07F,	Conteúdo [00000000]

REFERÊNCIA

Podemos considerar char a[8], a[7]='A',

a primeira célula	0x00F,	Conteúdo [00000000]
...a segunda	0x01F,	Conteúdo [00000000]
...a terceira	0x02F,	Conteúdo [00000000]
...a quarta	0x03F,	Conteúdo [00000000]
...a quinta	0x04F,	Conteúdo [00000000]
...a sexta	0x05F,	Conteúdo [00000000]
...a sétima	0x06F,	Conteúdo [00000000]
...a oitava	0x07F,	Conteúdo [01000001]

REFERÊNCIA

Quando fazemos referência, estamos obtendo exatamente este valor, que é o endereço da célula na memória.

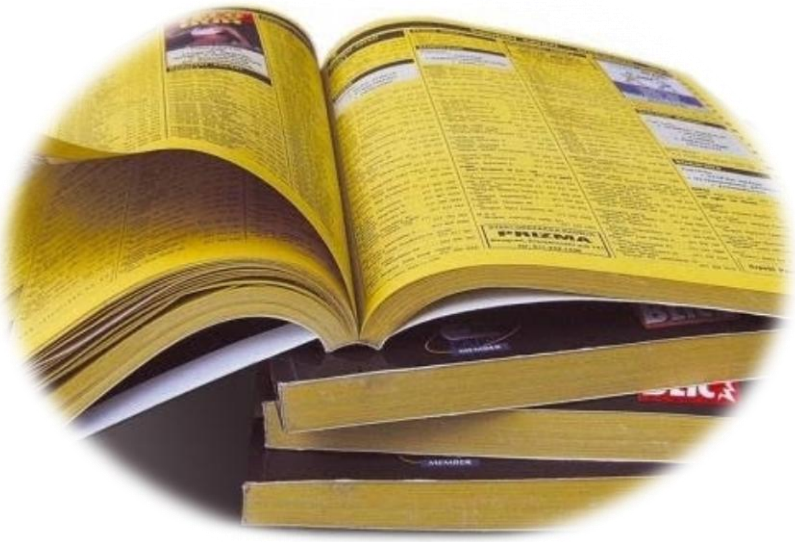
A referência é dada pelo operador &.

DEREFERÊNCIA

Quando um ponteiro aponta para um endereço de memória, a operação para acessar o conteúdo do endereço apontado é chamado de **dereferência**.

O operador unário * é usado para fazer a dereferência.

Note que este uso do símbolo * não tem relação com o símbolo de multiplicação.



OPERAÇÕES COM REFERÊNCIA/DEFERÊNCIA

Quando incrementamos um ponteiro ele passa a apontar para o próximo valor do mesmo tipo para o qual o ponteiro aponta.

O decremento funciona semelhantemente. Supondo que **p** é um ponteiro, as operações são escritas como:

p++; ou p--;

```
#include <stdio.h>
#include <stdlib.h>
main()
{
 int a[3]= {10, 15, 9};
 int *p;
 p= a; // ou p = &p[0];
  printf ("Valor de a na posicao 0 %d\n", *p);
 p++;
 printf ("Valor de a na posicao 1 %d\n", *p);
 p++;
 printf ("Valor de a na posicao 2 %d\n", *p);
 p++;
}
```

OPERAÇÕES COM REFERÊNCIA/DEFERÊNCIA

Para incrementar o conteúdo da variável apontada pelo ponteiro **p**, faz-se:

```
(*p)++;
```

Outras operações aritméticas úteis são a soma e subtração de inteiros com ponteiros. **Para incrementar um ponteiro** de 15 (o destino do ponteiro). Basta fazer:

```
p=p+15;    ou    p+=15;
```

E se você quiser **usar o conteúdo** do ponteiro 15 posições adiante:

```
*(p+15);
```

A subtração funciona da mesma maneira.

OPERAÇÕES COM REFERÊNCIA/DEFERÊNCIA

```
#include <stdio.h>
#include <stdlib.h>

main()
{
 int a[3]= {10, 15, 9};

 int *p;

 p= a; // ou p = &p[0];

 //somando o valor 20 na posicao a[0]
 (*p)= (*p)+20;

 //somando o valor 30 na posicao a[2]
 *(p+2)= *(p+2)+30;


 printf ("Valor de a na posicao 0 %d\n", *p);

 p++;

 printf ("Valor de a na posicao 1 %d\n", *p);

 printf ("Valor de a na posicao 2 %d\n", *(p+1));

}
```

VETORES COMO PONTEIROS

Na declaração da matriz: **int mat[10][10]** o compilador C calcula o tamanho, em bytes, necessário para armazenar esta matriz.

Este tamanho é: **tam1 + tam2 + tam3 + ... + tamN * tamanho_do_tipo**

O compilador aloca este número de bytes em um espaço livre de memória.

O ***nome da variável*** que você declarou é na verdade ***um ponteiro*** *para o tipo da variável da matriz*.

Durante a alocação na memória o espaço para a matriz, o nome da variável (que é um ponteiro) e aponta para o *primeiro* elemento da matriz.

Então como é que podemos usar a seguinte notação?

mat[5] o que é igual a ***(mat+5)**

VETORES COMO PONTEIROS

Todo vetor começa com indexação zero **É porque**, ao pegarmos o valor do primeiro elemento de um vetor, queremos, de fato, ***nome_da_variável** e então devemos ter um índice igual a zero.

Assim sabemos que:

`*nome_da_variável` é equivalente a `nome_da_variável[0]`

Podemos ter índices negativos. Estaríamos pegando posições de memória antes do vetor. O compilador C não verifica a validade dos índices.

A linguagen C *não* sabe o tamanho do vetor.

Ele apenas aloca a memória, ajusta o ponteiro do nome do vetor para o início do mesmo e, quando você usa os índices, encontra os elementos requisitados.

VARREDURA SEQUENCIAL DE UMA MATRIZ

Quando temos que varrer todos os elementos de uma matriz de uma forma sequencial, podemos usar um ponteiro, o qual vamos incrementando.

Qual a vantagem?
Considere o seguinte programa para zerar uma matriz:

```
int main ()
{       float matrx [50][50];
int i,j;
        for (i=0;i<50;i++)
                for (j=0;j<50;j++)
                        matrx[i][j]=0.0;
        return(0);
}
```

O programa, cada vez que se faz **matrx[i][j]** o programa tem que calcular o deslocamento para dar ao ponteiro. Ou seja, o programa tem que calcular 2500 deslocamentos.

VARREDURA SEQUENCIAL DE UMA MATRIZ

Podemos reescrevê-lo usando ponteiros:

```
int main ()
{
        float matrx [50][50];
        float *p;
        int count;
        p=matrx[0];
        for (count=0;count<2500;count++)
      {
        *p=0.0;
               p++;
        }
        return(0);
}
```

No segundo programa o único cálculo que deve ser feito é o de um incremento de ponteiro. Fazer 2500 incrementos em um ponteiro é muito mais rápido que calcular 2500 deslocamentos completos.

OBS.: um ponteiro é uma variável, mas o nome de um vetor não é uma variável.
Isto significa, que não se consegue alterar o endereço que é apontado pelo "nome do vetor".

PONTEIROS COMO VETORES

Considerando que o nome de um vetor é um ponteiro constante e, também podemos indexar o nome de um vetor, como conseqüência podemos também indexar um ponteiro qualquer.

```
#include <stdio.h>
int main ()
{
   int matrix [10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
   int *p;
   p=matrix;
   printf ("\nO terceiro elemento do vetor e: %d",p[2]);
   printf ("\nO terceiro elemento do vetor e: %d",*(p+2));
   return(0);
}
```

Podemos ver que **p[2]** equivale a ***(p+2)**.

INSTITUTO FEDERAL
RIO GRANDE DO SUL

Alocação Dinâmica

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

ALOCAÇÃO DINÂMICA

A alocação dinâmica permite alocar memória em tempo de execução, ou seja, alocar memória para novas variáveis ou redimensionar as existentes quando o programa está sendo executado.

O padrão C ANSI define apenas 4 funções para o sistema de alocação dinâmica, disponíveis na biblioteca **stdlib.h**:

- **malloc**
- **calloc**
- **realloc**
- **free**

MALLOC

A função **malloc()** serve para alocar memória e tem o seguinte protótipo:

```
void *malloc (unsigned int num);
```

A função toma o número de bytes que queremos alocar (**num**), aloca na memória e retorna um ponteiro **void *** para o primeiro byte alocado.

O ponteiro **void *** pode ser atribuído a qualquer tipo de ponteiro.

Se não houver memória suficiente para alocar a memória requisitada a função **malloc()** retorna um ponteiro nulo.

MALLOC

```
main (void)
{
      int *p;
      float a = 100000000;   /* Determina o valor de a */
            p= (int *)malloc (a * sizeof(float));
            if (!p)
                  printf ("** Erro: Memória Insuficiente **");
            else
                  printf ("Memória alocada som sucesso!!!");
       system("pause");
       return 0;
}
```

MALLOC

No exemplo acima, é alocada memória suficiente para se colocar **a (“a” vezes)** números float.

O operador **sizeof()** retorna o número de bytes de um inteiro.

Ele é util para se saber o tamanho de tipos.

O ponteiro **void*** que **malloc()** retorna é convertido para um **int*** pelo cast e é atribuído a **p**.

A declaração seguinte testa se a operação foi bem sucedida. Se não tiver sido, **p** terá um valor nulo, o que fará com que **!p** retorne verdadeiro.

Se a operação tiver sido bem sucedida, podemos usar o vetor de inteiros alocados normalmente, por exemplo, indexando de **p[0]** a **p[(a-1)]**.

CALLOC

A função **calloc()** também serve para alocar memória, mas possui um protótipo um pouco diferente:

```
void *calloc (unsigned int num, unsigned int size);
```

A função aloca uma quantidade de memória igual a **num * size**, isto é, aloca memória suficiente para uma matriz de **num** objetos de tamanho **size**.

Retorna um ponteiro **void *** para o primeiro byte alocado.

O ponteiro **void *** pode ser atribuído a qualquer tipo de ponteiro.

Se não houver memória suficiente para alocar a memória requisitada a função **calloc()** retorna um ponteiro nulo.

REALLOC

A função **realloc()** serve para realocar memória e tem o seguinte protótipo:

```
void *realloc (void *ptr, unsigned int num);
```

A função modifica o tamanho da memória previamente alocada apontada por ***ptr** para aquele especificado por **num**.

O valor de **num** pode ser maior ou menor que o original.

Um ponteiro para o bloco é devolvido porque **realloc()** pode precisar mover o bloco para aumentar seu tamanho.

Se isso ocorrer, o conteúdo do bloco antigo é copiado no novo bloco, e nenhuma informação é perdida.

Se **ptr** for nulo, aloca **size** bytes e devolve um ponteiro; se **size** é zero, a memória apontada por **ptr** é liberada.

Se não houver memória suficiente para a alocação, um ponteiro nulo é devolvido e o bloco original é deixado inalterado.

FREE

Quando alocamos memória dinamicamente é necessário que nós a liberemos quando ela não for mais necessária. Para isto existe a função **free()** cujo protótipo é:

```
void free (void *p);
```

Basta então passar para **free()** o ponteiro que aponta para o início da memória alocada.

INSTITUTO FEDERAL
RIO GRANDE DO SUL

Alocação de Memória
Dados Primitivos

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

ALOCAÇÃO DE MEMÓRIA

Consiste no processo de solicitar/utilizar memória durante o processo de execução de uma aplicação.

A alocação de memória no computador pode ser dividida em dois grupos principais:

- **Alocação Estática**
- **Alocação Dinâmica**

ALOCAÇÃO DE MEMÓRIA

Alocação Estática: os dados tem um tamanho fixo e estão organizados seqüencialmente na memória do computador.

Um exemplo típico de alocação estática são as variáveis globais e arrays;

ALOCAÇÃO DE MEMÓRIA

Alocação Dinâmica: os dados não precisam ter um tamanho fixo, pois podemos definir para cada dado quanto de memória que desejamos usar.

Sendo assim vamos alocar espaços de memória (blocos) que não precisam estar necessariamente organizados de maneira seqüencial, podendo estar distribuídos de forma esparsa na memória do computador.

Na alocação dinâmica, vamos pedir para alocar/desalocar blocos de memória, de acordo com a nossa necessidade, reservando ou liberando blocos de memória durante a execução de um programa.

Para poder "achar" os blocos esparsos na memória usamos as variáveis do tipo Ponteiro (indicadores de endereços de memória).

TIPOS DE DADOS

TIPOS DE DADOS PRIMITIVOS

A cada variável está associado um Tipo de Dados. O tipo de dado define quais os valores que a variável pode conter. Se, por exemplo, dissermos que uma variável é do tipo Inteiro, não poderemos lá colocar um valor Real ou um Caractere.

Ao declararmos o tipo de dados de uma variável, estamos a definir, não só, o tipo de valores que esta pode conter, mas também quais as operações que com elas podemos realizar.

Exemplo de tipos primitivos:

- INT
- REAL
- CHAR
- BOOL

TIPOS DE DADOS

TIPOS DE DADOS COMPOSTOS HOMOGÊNEOS

O nome mais comum atribuído as variáveis compostas homogêneas é **vetor ou matrizes**.

Vetores, matrizes ou *arrays* correspondem a um tipo de dado utilizado para representar (armazenar e manipular) uma coleção de valores do mesmo tipo.

Uma outra definição para vetores ou matrizes corresponde a **um conjunto de variáveis, do mesmo tipo**, identificadas por um único nome, onde cada variável (posição) é referenciada por meio de número chamado de índice. **Os colchetes são utilizados para conter o índice.**

TIPOS DE DADOS

TIPOS DE DADOS COMPOSTOS HOMOGÊNEOS

As variáveis compostas homogêneas são estruturas de dados que caracterizam-se por um conjunto de variáveis do mesmo tipo. Elas pode ser unidimensionais ou multidimensionais.

Unidimensionais (*vetores*)
A variável composta homogênea unidimensional caracteriza-se por dados agrupados linearmente numa única direção, como uma linha reta.

Multidimensionais (matrizes)
A variável composta multidimensional caracteriza-se por dados agrupados em diferentes direções, como em um cubo;

TIPOS DE DADOS

TIPOS DE DADOS COMPOSTOS HETEROGÊNEOS:

As estruturas heterogêneas são conjuntos de dados formados por **tipos de dados primitivos diferentes** (campos do registro) em uma mesma estrutura.

REGISTROS

Estrutura de Dados – STRUCT

As estruturas de dados consistem em criar apenas um dado que contém vários membros, que nada mais são do que outras variáveis.

De uma forma mais simples, é como se uma variável tivesse outras variáveis dentro dela.

A vantagem em se usar estruturas de dados é que podemos agrupar de forma organizada vários tipos de dados diferentes, por exemplo, dentro de uma estrutura de dados podemos ter juntos tanto um tipo float, um inteiro, um char ou um double.

As variáveis que ficam dentro da estrutura de dados são chamadas de membros.

```
struct nome_da_estrutura { tipo_de_dado nome_do_membro; };
```

REGISTROS

Ponteiro de Struct

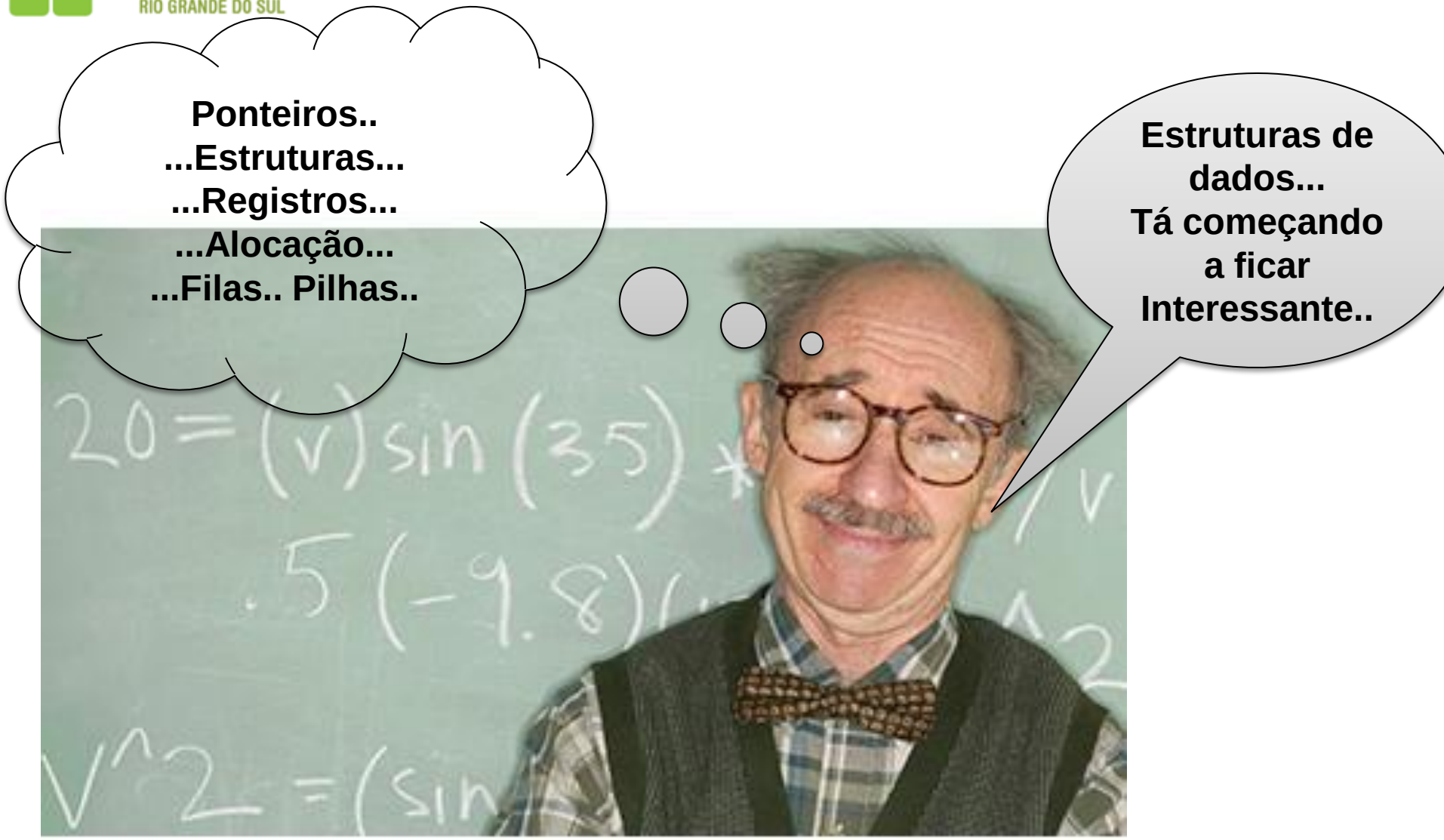
Uma *struct* consiste em vários dados agrupados em apenas um.

Para acessarmos cada um desses dados, usamos um ponto (.) para indicar que o nome seguinte é o nome do membro.

Um ponteiro guarda o endereço de memória que pode ser acessado diretamente.

Observação: Também pode ser usado a seta (–>) para indicar a propriedade.

INSTITUTO FEDERAL
RIO GRANDE DO SUL



Entenda... Mostrando o endereço

```
#include <stdlib.h>
#include <stdio.h>

main()
{
     float n=0;
     n = 2015;
     printf("Valor de n: %f \n", n);
     printf("Endereco de n: %x \n", &n);

 }
```

Entenda... Inicializando um ponteiro

```
#include <stdlib.h>
#include <stdio.h>
main()
{
        int n=0;
        int *pn = &n;
        n = 2015;
        printf("Valor de n: %d \n", n);
        printf("Endereco de n: %x \n", &n);
        printf("Conteudo do Ponteiro de n: %d \n", *pn);
        printf("Ponteiro de n: %x \n", pn);
}
```

Entenda... Mostrando o endereço x valor

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int *a, *b;
   a = (int * ) malloc ( sizeof(int) * 100);
   b = (int * ) malloc ( sizeof(int) * 3 );

    for (int i = 0; i< 100; i++)
    {
        a[i] = 0;
    }

    b[0] = 0;
    b[1] = 1;
    b[2] = 2;

    printf("valor  %d  - endereco %p \n",a[0] , &a[0]);
    printf("valor  %d  - endereco %p \n", a[1], &a[1]);
    printf("valor  %d  - endereco %p \n", a[2], &a[2]);
    printf("valor  %d  - endereco %p \n", b[0], &b[0]);
    printf("valor  %d  - endereco %p \n", b[1], &b[1]);
    printf("--> a  %d\n", sizeof *a);
    system("pause");
    return(0);
}
```

1) Qual é o valor de a[4] e &a[4] ?
2) Qual sua diferença b[2] e &b[2] ?

Entenda... Vetor de Registros

```
#include <stdio.h>
#include <string.h>

int main(void)
{

    Cpessoa aluno[4];

    strcpy(aluno[0].nome, "Maria");
    aluno[0].idade = 22;

    strcpy(aluno[1].nome, "Augusto");
    aluno[1].idade = 22;

    strcpy(aluno[2].nome, "Julia");
    aluno[2].idade = 23;

    strcpy(aluno[3].nome, "Alberto");
    aluno[3].idade = 24;

    for (int i = 0; i<4 ; i++)
     {
       printf("%s  - %d \n", aluno[i].nome, aluno[i].idade);
      }
     getchar();
}
```

```
typedef struct Cpessoa
{
        char nome[20];
        int idade;
};
```

Entenda... Vetor de Registros - Parte 2

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int *a, *b;
   a = (int * ) malloc ( sizeof(int) * 100);
   b = (int * ) malloc ( sizeof(int) * 3 );

    for (int i = 0; i< 100; i++)
    {
        a[i] = i;
    }

    b[0] = 0;
    b[1] = 1;
    b[2] = 2;

    printf("valor  %d  - endereco %p \n", a[0] ,&a[0]);
    printf("valor  %d  - endereco %p \n", a[1], &a[1]);
    printf("valor  %d  - endereco %p \n", a[2], &a[2]);
    printf("valor  %d  - endereco %p \n", b[0], &b[0]);
    printf("valor  %d  - endereco %p \n", b[1], &b[1]);
    printf("--> a  %d\n", sizeof *a);
    system("pause");
    return(0);
}
```

1) Qual será o valor de a[3] e &a[3] ?
2) Qual sua diferença a[3] e &a[3] ?

Entenda... Usando vetor x ponteiro

```
for (int i = 0; i<4 ; i++)
    {
      printf("%s  - %d \n", aluno[i].nome, aluno[i].idade);
     }
     getchar();
```

```
printf("%s  - %d \n", (*paluno).nome, (*paluno).idade);
```

Ou

```
printf("%s  - %d \n", paluno->nome, paluno->idade);
```

Entenda... Inicializando Vetor

```
#include <stdio.h>
#include <string.h>

typedef struct Cpessoa
{
        char nome[20];
        int idade;
} ;

int main(void)
{
    Cpessoa aluno[4];

    strcpy(aluno[0].nome, “Homer");
    aluno[0].idade = 22;
    strcpy(aluno[1].nome, “Marge");
    aluno[1].idade = 22;
    strcpy(aluno[2].nome, “Lisa");
    aluno[2].idade = 23;
    strcpy(aluno[3].nome, “Bart");
    aluno[3].idade = 24;

    for (int i = 0; i<4 ; i++)
    {
     printf("%s  - %d \n", aluno[i].nome, aluno[i].idade);
     }
    getchar();
    return 1;
}
```

Entenda... Inicializando Ponteiro

```
#include <stdio.h>
#include <string.h>

typedef struct Cpessoa
{

        char nome[20];
        int idade;
} ;

int main(void)
{

    Cpessoa aluno;
    Cpessoa *paluno;
    paluno = &aluno;

    strcpy((*paluno).nome, "Homer");
    (*paluno).idade = 22;

     printf("%s  - %d \n", (*paluno).nome, (*paluno).idade);

     getchar();
return 1;
}
```

Entenda... Alocando Dinamicamente

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
struct Cpessoa
{       char nome[20];
         int idade;} ;

int main(void)
{
    Cpessoa *paluno;
    paluno = (struct Cpessoa *) malloc(4*sizeof(struct Cpessoa));

    strcpy((*paluno).nome, "Homer");
    (*paluno).idade = 22;
    strcpy((*(paluno+1)).nome, "Marge");
    (*(paluno+1)).idade = 22;
    strcpy((*(paluno+2)).nome, "Lisa");
    (*(paluno+2)).idade = 23;
    strcpy((*(paluno+3)).nome, "Bart");
    (*(paluno+3)).idade = 24;
    for (int i = 0; i<4 ; i++)
    {
     printf("%s  - %d \n", (*(paluno+i)).nome, (*(paluno+i)).idade);
     }
     getchar();
return 1;
}
```

Desafio

Altere o programa de forma que leia valores enquanto não informado o valor de saída. Vamos acordar que para cada inserção, deverá perguntar:
“Inserir Novo: S/N”.

Ao final, mostrar todos.